

Основы Операционных Систем

МФТИ-2026

Тема 6

Планирование процессов

Планирование процессов

Уровни планирования

- Долгосрочное планирование – планирование заданий
- Среднесрочное планирование – swapping
- Краткосрочное планирование – планирование использования процессора.

Планирование процессов

Цели планирования

- Справедливость
- Эффективность
- Сокращение полного времени выполнения (turnaround time)
- Сокращение времени ожидания (waiting time)
- Сокращение времени отклика (response time)

Планирование процессов

Желаемые свойства алгоритмов

- Предсказуемость
- Минимизация накладных расходов
- Равномерность загрузки вычислительной системы
- Масштабируемость

Планирование процессов

Параметры планирования

- Статические параметры вычислительной системы – например, предельные значения ее ресурсов.
- Статические параметры процесса – кем запущен, степень важности, запрошенное процессорное время, какие требуются ресурсы и т.д.

Статические

- Динамические параметры вычислительной системы – например, количество свободных ресурсов в данный момент.
- Динамические параметры процесса – текущий приоритет, размер занимаемой оперативной памяти, использованное процессорное время и т.д.

Динамические

Планирование процессов

Важные динамические параметры процесса



Планирование процессов

Вытесняющее и невытесняющее

- Перевод процесса из состояния «*исполнение*» в состояние «*закончил исполнение*»
- Перевод процесса из состояния «*исполнение*» в состояние «*ожидание*»

Вынужденное принятие решения

- Перевод процесса из состояния «*исполнение*» в состояние «*готовность*»
- Перевод процесса из состояния «*ожидание*» в состояние «*готовность*»

Невынужденное принятие решения

Планирование процессов

Вытесняющее и невытесняющее

- Перевод процесса из состояния «*исполнение*» в состояние «*закончил исполнение*»
- Перевод процесса из состояния «*исполнение*» в состояние «*ожидание*»

Вынужденное принятие решения

- Перевод процесса из состояния «*исполнение*» в состояние «*готовность*»
- Перевод процесса из состояния «*ожидание*» в состояние «*готовность*»

Невынужденное принятие решения

Принятие только вынужденных решений –
невытесняющее планирование

Планирование процессов

Вытесняющее и невытесняющее

- Перевод процесса из состояния «*исполнение*» в состояние «*закончил исполнение*»
- Перевод процесса из состояния «*исполнение*» в состояние «*ожидание*»

Вынужденное принятие решения

- Перевод процесса из состояния «*исполнение*» в состояние «*готовность*»
- Перевод процесса из состояния «*ожидание*» в состояние «*готовность*»

Невынужденное принятие решения

Принятие вынужденных и невынужденных решений –
вытесняющее планирование

Алгоритмы планирования

FCFS (First Come – First Served)

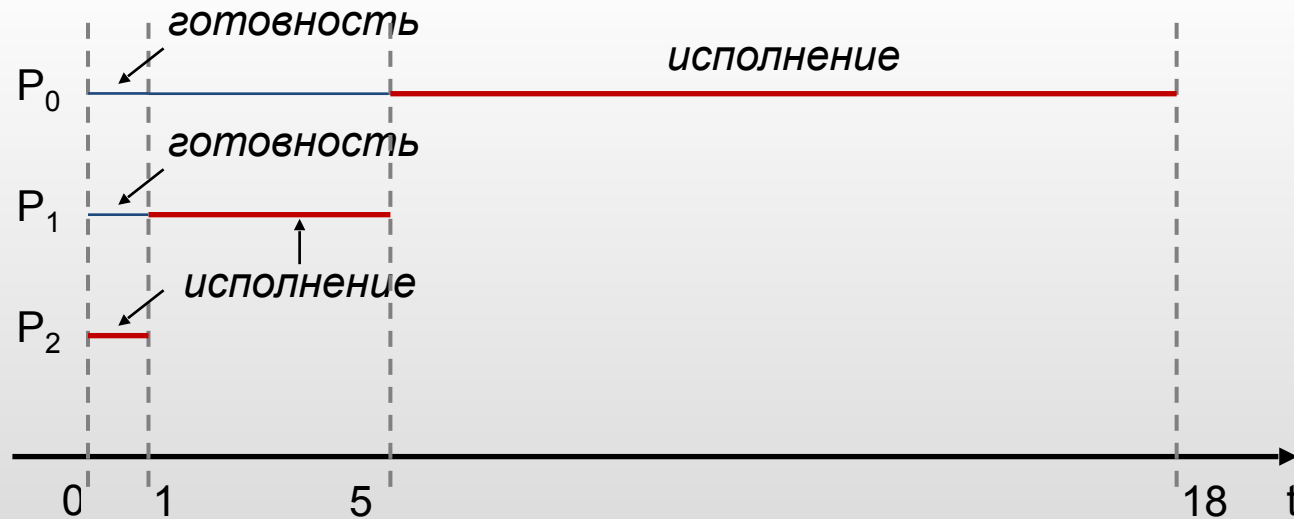
Процессы	P_0	P_1	P_2
Продолжительность CPU burst	13	4	1



Алгоритмы планирования

FCFS (First Come – First Served)

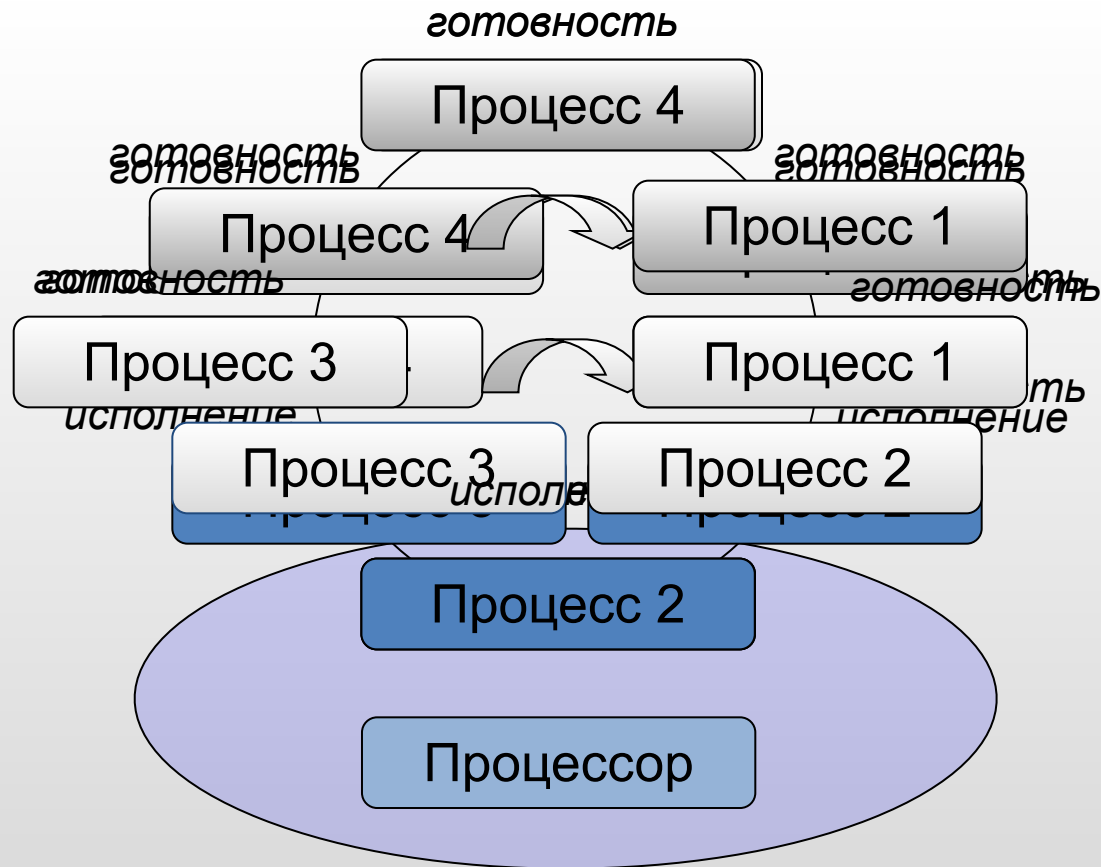
Процессы	P_2	P_1	P_0
Продолжительность CPU burst	1	4	13



$$tt = \frac{18 + 5 + 1}{3} = 8$$

$$wt = \frac{5 + 1 + 0}{3} = 2$$

RR (Round Robin)



Алгоритмы планирования

RR (Round Robin)

Остаток времени CPU burst $\leq$ кванта времени:

- процесс освобождает процессор до начала нового кванта;
- на исполнение выбираем новый процесс из начала очереди готовых и выделяем ему новый квант времени.

Остаток времени CPU burst $>$ кванта времени:

- по окончании кванта процесс помещается в конец очереди готовых к исполнению процессов;
- на исполнение выбираем новый процесс из начала очереди готовых.

Алгоритмы планирования

RR (Round Robin)

Процессы	P ₀	P ₁	P ₂
Продолжительность CPU burst	13	4	1

Величина кванта времени – 4

время	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
P ₀	И	И	И	И	Г	Г	Г	Г	Г	И	И	И	И	И	И	И	И	И
P ₁	Г	Г	Г	Г	И	И	И	И										
P ₂	Г	Г	Г	Г	Г	Г	Г	Г	И									

Исполнение

P₀

Очередь готовых

P ₀	P ₀	P ₀
----------------	----------------	----------------

$$tt = \frac{18 + 8 + 9}{3} = 11\frac{2}{3}$$

$$wt = \frac{5 + 4 + 8}{3} = 5\frac{2}{3}$$

Алгоритмы планирования

RR (Round Robin)

Процессы	P ₀	P ₁	P ₂
Продолжительность CPU burst	13	4	1

Величина кванта времени – 1

время	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
P ₀	И	Г	Г	И	Г	И	Г	И	Г	И	И	И	И	И	И	И	И	И
P ₁	Г	И	Г	Г	И	Г	И	Г	И									
P ₂	Г	Г	И															

Исполнение

Очередь готовых

$$tt = \frac{18 + 9 + 3}{3} = 10$$

$$wt = \frac{5 + 5 + 2}{3} = 4$$

P₀

P₀

P₀

P₀

Алгоритмы планирования

SJF (Shortest Job First) невытесняющий

Процессы	P ₀	P ₁	P ₂	P ₃
Продолжительность CPU burst	5	3	7	1

время	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
P ₀	Г	Г	Г	Г	И	И	И	И	И							
P ₁	Г	И	И	И												
P ₂	Г	Г	Г	Г	Г	Г	Г	Г	Г	И	И	И	И	И	И	И
P ₃	И															

ИСПОЛНЕНИЕ

ГОТОВНОСТЬ

$$tt = \frac{9 + 4 + 16 + 1}{4} = 7\frac{1}{2}$$

$$wt = \frac{4 + 1 + 9 + 0}{4} = 3\frac{1}{2}$$

P₀

P ₀	P ₁	P ₂	P ₃
----------------	----------------	----------------	----------------

Алгоритмы планирования

SJF (Shortest Job First) вытесняющий

Процессы	P ₀	P ₁	P ₂	P ₃
Продолжительность CPU burst	6	2	5	5
Момент появления в очереди	0	2	6	0

время	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
P ₀	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	И	И	И	И	И	И
P ₁			И	И														
P ₂							Г	И	И	И	И	И						
P ₃	И	И	Г	Г	И	И	И											

исполнение

готовность

$$tt = \frac{18 + 2 + 6 + 7}{4} = 8\frac{1}{4}$$

$$wt = \frac{12 + 0 + 1 + 2}{4} = 3\frac{3}{4}$$

P₀

P₀

P₁

P₂

P₃

Алгоритмы планирования

SJF (Shortest Job First) приближение

$\tau(n)$ – величина n -го CPU burst
 $T(n+1)$ – предсказание для $n+1$ -го CPU burst
 α – параметр от 0 до 1

$$T(n+1) = \alpha \tau(n) + (1 - \alpha)T(n),$$

$T(0)$ – произвольно

Если $\alpha = 0$, то $T(n+1) = T(n) = \dots = T(0)$, нет учета последнего поведения

Если $\alpha = 1$, то $T(n+1) = \tau(n)$, нет учета предыстории

Алгоритмы планирования

Гарантированное планирование

В системе разделения времени N пользователей:

T_i — время нахождения i -го пользователя в системе

τ_i — суммарное процессорное время процессов i -го пользователя

$\tau_i \ll T_i / N$ — пользователь обделен

$\tau_i \gg T_i / N$ — пользователю благоволят

$(\tau_i N) / T_i$ — коэффициент справедливости

На исполнение выбираются готовые процессы пользователя с наименьшим коэффициентом справедливости

Алгоритмы планирования

Приоритетное планирование

Каждому процессу процессор выделяется в соответствии с приписанным к нему числовым значением - приоритетом

Параметры для назначения приоритета бывают:

- внешние
- внутренние

Политика изменения приоритета:

- статический приоритет
- динамический приоритет

Алгоритмы планирования

Приоритетное планирование невытесняющее

Процессы	P ₀	P ₁	P ₂	P ₃
Продолжительность CPU burst	6	2	5	5
Момент появления в очереди	0	2	6	0
Приоритет	4	3	2	1

время	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
P ₀	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	И	И	И	И	И	И
P ₁			Г	Г	Г	И	И											
P ₂							Г	И	И	И	И	И						
P ₃	И	И	И	И	И													

$$tt = \frac{18 + 5 + 6 + 5}{4} = 8\frac{1}{2}$$

$$wt = \frac{12 + 3 + 1 + 0}{4} = 4$$

исполнение

P₀

готовность

P₀

P₁

P₂

P₃

Алгоритмы планирования

Приоритетное планирование вытесняющее

Процессы	P ₀	P ₁	P ₂	P ₃
Продолжительность CPU burst	6	2	5	5
Момент появления в очереди	0	2	6	0
Приоритет	4	3	2	1

время	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
P ₀	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	И	И	И	И	И	И
P ₁			Г	Г	Г	И	Г	Г	Г	Г	Г	И						
P ₂							И	И	И	И	И							
P ₃	И	И	И	И	И													

$$tt = \frac{18 + 10 + 5 + 5}{4} = 9\frac{1}{2}$$

$$wt = \frac{12 + 8 + 0 + 0}{4} = 5$$

исполнение

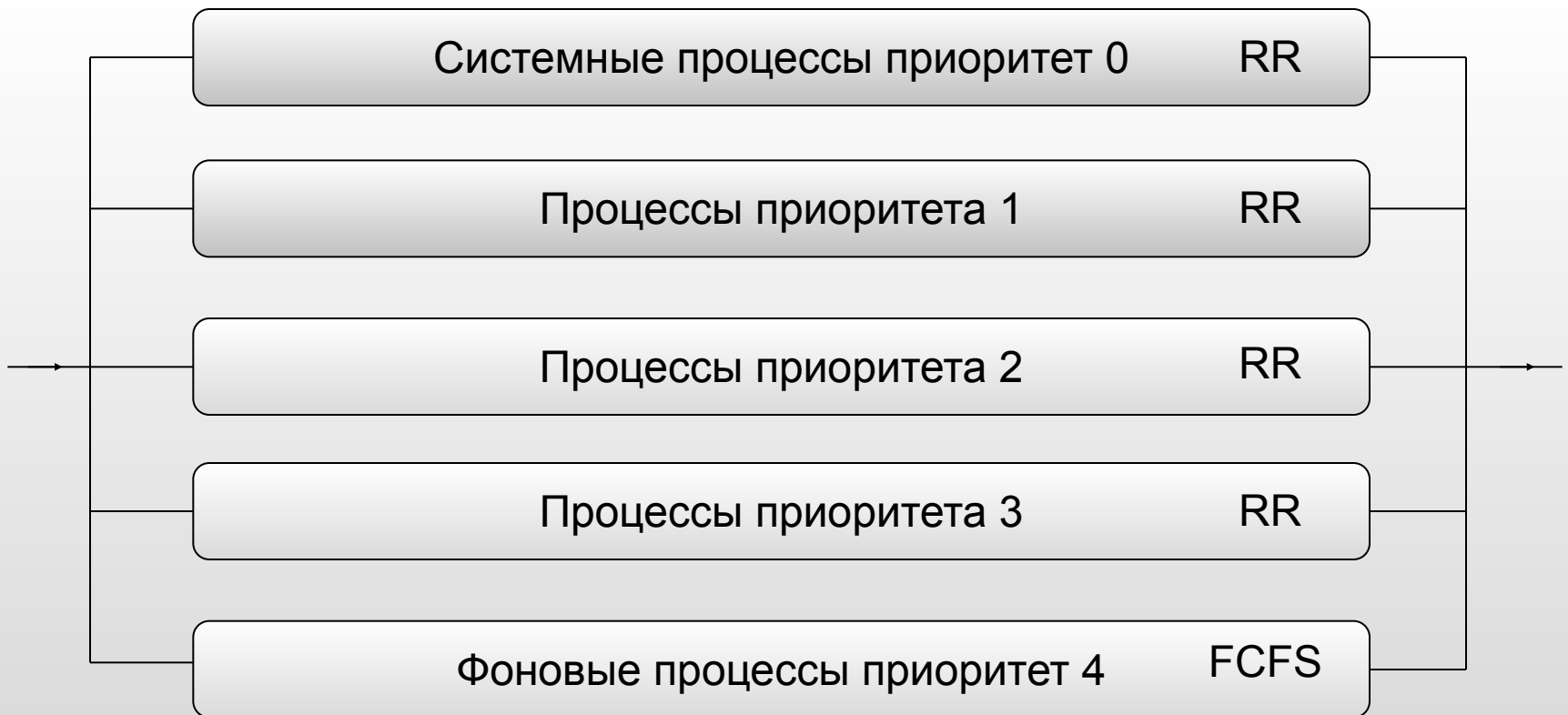
P₀

готовность

P ₀	P ₁	P ₂	P ₃
----------------	----------------	----------------	----------------

Алгоритмы планирования

Многоуровневые очереди (Multilevel Queue)



Алгоритмы планирования

Многоуровневые очереди с обратной связью (Multilevel Feedback Queue)



Алгоритмы планирования

Многоуровневые очереди с обратной связью (Multilevel Feedback Queue)

Для полного описания необходимо задать:

- количество очередей в состоянии готовность
- алгоритм планирования между очередями
- алгоритмы планирования внутри очередей
- куда помещается родившийся процесс
- правила перевода процессов из одной очереди в другую

Планирование процессов в Linux

Эволюция планировщика по умолчанию

- $O(1)$ (до v.2.6.xx — основной)
- *CFS* (v.2.6.xx - v.6.6.xx — основной)
- *EEVDF* (с v.6.6.xx — основной)

Планирование процессов в Linux

O(1) Scheduler (O(1))

- Планирование происходит по квантам виртуального времени. На исполнение выбирается процесс с наибольшим приоритетом из очереди активных
- Когда очередь активных пустеет, очередь меняют местами с очередью исчерпавших квант
- Для повышения интерактивности к неактивным процессам прибавляют *sleep_avg* — среднее время сна — приоритет динамически подрастает (*recalc_task_prio(...)*)

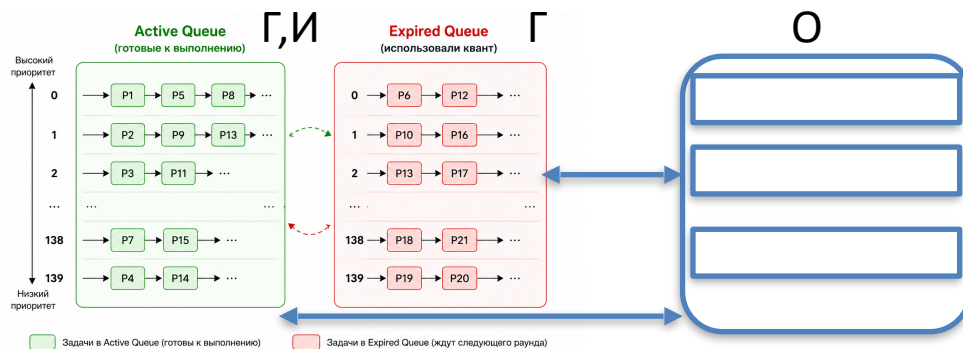


Рис. Очереди active и expired [ист]

Планирование процессов в Linux

Completely Fair Scheduler (CFS)

- Решает задачу «справедливой» выдачи виртуального времени потокам: $T_{\text{virtual}} \sim \text{const} |_{t \gg 1}$
- Оперирует переменной *vruntime* — сколько поток выполнялся с поправкой на приоритет
- На исполнение ставится процесс *min(vruntime)*
- *Информация о процессах - в RBTre по vruntime*

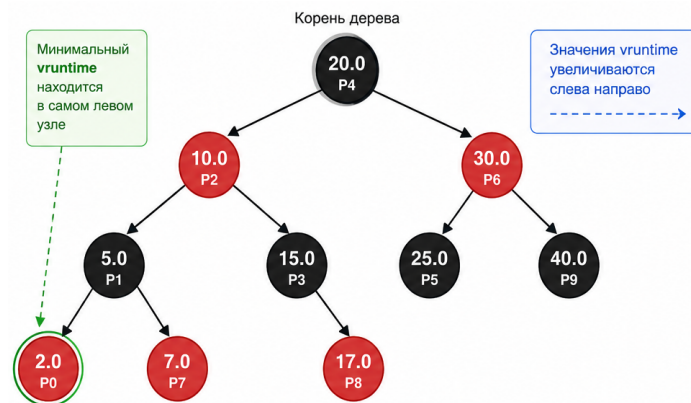


Рис. RBTre (vruntime) [\[ист.\]](#)

Планирование процессов в Linux

EEVDF

- Развивает CFS, вводя «eligibility» и «deadlines»
- *Планировщик определяет пул «eligible» задач, имеющих право выполняться ($lag \geq 0$ * — задача недополучила ресурсы)*
- Из пула выбирается задача с $\min(\text{Deadline})$
- *RBTree по deadline + heap ($lag \geq 0$)*

*$vruntime += actual_time + weight(0) / weight(NICE)$,
 $NICE: [-20, +19]$, $weight(NICE) \approx 1024 * (1.25)^{-NICE}$
 $lag = weight(NICE) * (V - v)$, V — средневзвеш. $T_{virtual}$
 $Deadline = vruntime + slice / weight(NICE)$*

*В алгоритме есть «хак» — задача может засыпать и сбрасывать $lag < 0$. Решается хакон «пометить задачу как *sched_delayed*» — *vruntime* продолжит расти.

Планирование процессов в Linux

Completely Fair Scheduler (CFS)

- Решает задачу «справедливой» выдачи виртуального времени потокам: $T_{\text{virtual}} \sim \text{const} |_{t \gg 1}$
- Оперирует переменной *vruntime* — сколько поток выполнялся с поправкой на приоритет
- На исполнение ставится поток *min(vruntime)*
- *Информация о процессах - в RBTre по vruntime*

$vruntime += \text{actual_time} + \text{weight}(0) / \text{weight}(\text{NICE}),$
 $\text{NICE}: [-20, +19], \text{weight}(\text{NICE}) \approx 1024 * (1.25)^{-\text{NICE}}$

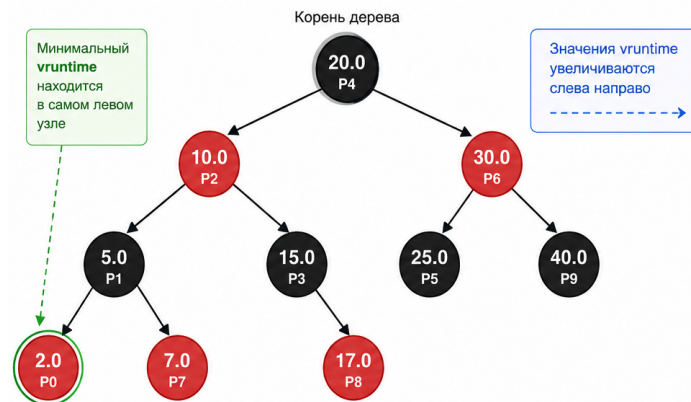


Рис. RBTre (vruntime) [\[ист.\]](#)

